Assistify - Intelligent Help Desk with RAG & Voice Engine

Actual Implementation Documentation (Code-Based)

Author: Jonathan (AAST Graduation Project)
Date: November 24, 2025
Version: 2.0

---

1. System Overview

1.1 What This System Actually Does

This project is helpdesk chatbot system that use voice input and artificial intelligence. User can ask question by typing or speaking, and system search in knowledge base to give accurate answer.

The main innovation is **TOON format** (Token-Oriented Object Notation) that reduce token usage by 40-60% compared to JSON, making LLM response faster.

1.2 Real Architecture (Not Theory)

System have **3 separate servers** running on different ports:

```
┌──────────────────────┐
│  Login Server        │ ← User face this (Port 7001)
│  (login_server.py)   │
└──────────────────────┘
           │
           │
           ↓ WebSocket Proxy
┌──────────────────────┐
│  RAG Server          │ ← Voice + RAG (Port 7000)
│  (assistify_rag_     │
│   server.py)         │
└──────────────────────┘
           │
           │
           ↓ HTTP Request
┌──────────────────────┐
│  LLM Server          │ ← Pure inference (Port 8000)
│  (main_llm_          │
│   server.py)         │
└──────────────────────┘
```

Why 3 servers?

- **Login Server (7001)**: Handle authentication, session, admin panel, file upload. Act as WebSocket proxy to connect frontend to RAG server.

- **RAG Server (7000)**: Process voice input using faster-whisper, search documents in ChromaDB, format context using TOON, send to LLM.
- **LLM Server (8000)**: Only do inference using Qwen model on GPU. No other task. Keep it simple for performance.

1.3 Technologies Actually Used (From Code)

Backend:

- FastAPI (all 3 servers use this framework)
- Python 3.10+
- SQLite3 for database (conversations, users, analytics, sessions)

AI Models:

- **LLM**: Qwen2.5-7B-Instruct Q4_K_M (quantized, split in 2 GGUF files)
  - Location: `backend/Models/Qwen2.5-7B-LLM/`
  - Size: 4.36 GB total
  - Inference library: llama-cpp-python with CUDA support
- **Voice Recognition**: faster-whisper medium.en
  - Location: `backend/Models/models--Systran--faster-whisper-medium.en/`
  - Run on CUDA GPU with float16
  - Have VAD (Voice Activity Detection) built-in
- **Embeddings**: all-MiniLM-L6-v2 (sentence-transformers)
  - Used for convert text to vector for ChromaDB

GPU Requirements:

- NVIDIA GPU with CUDA support (tested on RTX 3070 8GB)
- CUDA Toolkit installed
- llama-cpp-python built with CUBLAS flag

Vector Database:

- ChromaDB (persistent storage)
- Store document embeddings for semantic search
- Location: `backend/chroma_db/`

Authentication:

- Password hashing: bcrypt_sha256 + pbkdf2_sha256 (backward compatible)
- Session management: itsdangerous URLSafeSerializer
- OAuth: Google OAuth 2.0 (using authlib)
- OTP: EmailJS for sending verification code

---

2. How Each Server Works (Based on Actual Code)

2.1 LLM Server (main_llm_server.py)

Purpose: Only run Qwen model inference. Nothing else.

Startup Process (from code line 119-210):

1. Check if nvidia-smi available (verify GPU exists)
2. Check if llama-cpp-python built with CUDA support
3. Load Qwen model from `backend/Models/Qwen2.5-7B-LLM/`
4. Offload 10 layers to GPU (configurable in config.py: N_GPU_LAYERS=10)
5. Set context window to 512 tokens (N_CTX=512)
6. Set batch size to 2 (N_BATCH=2)
7. Start FastAPI on port 8000

GPU Configuration (from config.py):

```
N_GPU_LAYERS = 10        # How many layers on GPU (0-32)
N_CTX = 512              # Context window size
N_BATCH = 2              # Batch size for processing
ENFORCE_GPU = True       # Server will crash if GPU not available
```

API Endpoints (from code line 220-382):

1. **GET /health**
   - Check if LLM ready
   - Return GPU info (memory usage, layers loaded)
   - Response example:

```
{
  "status": "ready",
  "model": "qwen2.5-7b-instruct",
  "gpu_layers": 10,
  "context_size": 512
}
```

2. **POST /v1/chat/completions**
   - Main inference endpoint
   - Accept messages in OpenAI format:

```
{
  "model": "qwen2.5-7b-instruct",
  "messages": [
    {"role": "system", "content": "You are assistant"},
    {"role": "user", "content": "Hello"}
  ],
  "max_tokens": 80,
  "temperature": 0.7
}
```

- Process:
  1. Reset KV cache (llm.reset() - prevent memory buildup)
  2. Generate response using llama-cpp
  3. Return in OpenAI format
- Performance: ~24 seconds for greeting with RAG context

Important Code Logic (line 238-250):

```
# Before each request, reset KV cache
llm.reset()

# Cap max_tokens to prevent GPU OOM
max_tokens = min(request_max_tokens, 300)

# Generate
response = llm.create_chat_completion(
    messages=messages,
    max_tokens=max_tokens,
    temperature=temperature,
    stop=stop
)
```

2.2 RAG Server (assistify_rag_server.py)

Purpose: Handle voice input, search knowledge base, format context with TOON, call LLM.

Startup Process (from code line 121-192):

1. Initialize SQLite databases (conversations, sessions, analytics)
2. Create aiohttp session for LLM requests (with connection pooling)
3. Check CUDA available
4. Load faster-whisper model to GPU
5. Initialize ChromaDB collection "support_docs"
6. Start FastAPI on port 7000

faster-whisper Configuration (from code line 149-191):

```
whisper_model = WhisperModel(
    str(WHISPER_MODEL_PATH),
    device="cuda",              # Use GPU
    compute_type="float16",     # Fast GPU inference
    download_root=None          # Use local model only
)
```

Key Features Implemented:

A. Voice Processing Pipeline (WebSocket /ws)

How it work (from code line 625-814):

1. **Client connect WebSocket** → Server create connection_id (like "conn_a3f2b891")
2. **Browser send audio chunks** (binary data, PCM16 format, 16kHz sample rate)
3. **Server accumulate in buffer** (bytearray)
4. **Calculate energy of each chunk** to detect silence:

```
pcm_samples = np.frombuffer(audio, dtype=np.int16).astype(np.float32) /
32768.0
energy = np.sqrt(np.mean(pcm_samples ** 2))

if energy < 0.015:  # silence_threshold_energy
    silence_counter += 1
else:
    silence_counter = 0
```

5. **When detect 2.5 seconds silence** (40 consecutive chunks × 62.5ms = 2500ms):
   - Take all accumulated audio
   - Clear buffer
   - Send to faster-whisper for transcription
6. **Transcription** (code line 681-714):

```
segments, info = whisper_model.transcribe(
    pcm16,
    language="en",
    beam_size=5,
    vad_filter=True,      # Filter out silence parts
    vad_parameters=dict(
        min_silence_duration_ms=500,
        threshold=0.5
    ),
    no_speech_threshold=0.8  # Reject if 80%+ is silence
)
```

7. **Send transcript to client** via WebSocket
8. **Call LLM with RAG** (see next section)
9. **Send AI response back** to client

B. RAG Pipeline (call_llm_with_rag function)

Actual implementation (from code line 238-365):

1. **Check if greeting** (optimization to skip RAG):

```
greeting_patterns = ['hi', 'hello', 'hey', 'how are you', ...]
is_greeting = (
    len(text.strip().split()) <= 3 and
```

```
    any(pattern in text.lower() for pattern in greeting_patterns)
)
```

- If yes: Skip ChromaDB search (save 2-3 seconds)
- If no: Continue to step 2

2. **Search ChromaDB**:

```
relevant_docs = search_documents(text, top_k=1)  # Only 1 doc for speed
```

- Convert query to embedding using all-MiniLM-L6-v2
- Find most similar document in vector store
- Return document text

3. **Format context using TOON**:

```
doc_dicts = [{
    "page_content": doc_text,
    "metadata": {"doc_id": 0, "type": "support_info"}
}]
toon_context = format_rag_context_toon(doc_dicts)
```

- TOON format save 40-60% tokens vs JSON
- Example:

```
doc[1]:
content: How to reset password...
type: support_info
```

4. **Build system prompt** (ultra-minimal for speed):

```
system_prompt = f"Assistify assistant.{context}"
```

- Old version was ~200 tokens, now only ~5 tokens
- Context only added if RAG found documents

5. **Prepare messages**:

```
messages = [{"role": "system", "content": system_prompt}]
messages.extend(history[-1:])  # Only last 1 message from history
messages.append({"role": "user", "content": text.strip()})
```

6. **Call LLM server** via HTTP POST:

```
payload = {
    "model": "qwen2.5-7b-instruct",
    "messages": messages,
    "max_tokens": 80,      # Short answers for speed
    "temperature": 0.7,
    "stop": None
}
async with llm_session.post(LLM_URL, json=payload) as resp:
    result = await resp.json()
```

7. **Extract response and save**:

```
ai_text = result["choices"][0]["message"]["content"]
save_conversation(connection_id, text, ai_text, relevant_docs)
```

8. **Return response + document count**

Performance Optimization Done:

- Greeting detection: Skip RAG for simple greetings (save 20 seconds)
- Top-k=1: Only retrieve 1 document instead of 3 (save tokens)
- Max tokens=80: Shorter answers (save 15 seconds)
- History=1: Only keep last message (save tokens)
- TOON format: 40-60% less tokens than JSON
- Connection pooling: Reuse HTTP connection to LLM

Result: Response time reduced from 50 seconds to 10-15 seconds for RAG queries, 3-4 seconds for greetings.

C. Text Query Endpoint (POST /query)

Code location: Line 405-511

For user who don't want use voice, can send text directly:

```
@app.post("/query")
async def query_endpoint(
    request: Request,
    text: str = Form(...),
    user=Depends(require_login())
):
    connection_id = f"query_{uuid.uuid4().hex[:8]}"
    ai_text, docs = await call_llm_with_rag(text, connection_id, user)
    return {"response": ai_text, "sources": len(docs)}
```

Authentication required (must be logged in).

D. Document Upload (POST /upload)

Code location: Line 531-598

Admin can upload document to knowledge base:

1. Accept file (TXT or PDF)
2. Save to `backend/assets/`
3. Extract text:
    - TXT: Just read file
    - PDF: Use PyPDF2 to extract text from all pages
4. Generate document ID (like "upload_a3f2b891_document.pdf")
5. Create embedding and store in ChromaDB
6. Return success message

Note: PDF parsing can fail if PyPDF2 not installed. Will return error message but not crash.

2.3 Login Server (login_server.py)

Purpose: User authentication, session management, admin panel, WebSocket proxy.

Why this server exist?

- RAG server should only handle RAG logic
- Login server handle all user-related stuff
- Act as gateway between frontend and RAG server

Authentication Features Implemented:

A. User Registration (POST /register)

Code location: Line 1059-1148

Flow:

1. Check rate limit (3 requests per minute from same IP)
2. Validate input:
    - Username: 3-20 characters, alphanumeric + underscore
    - Password: 8-128 characters, must have uppercase, lowercase, digit, special char
    - Email: Valid email format
3. Hash password using bcrypt_sha256 (12 rounds)
4. Generate OTP code (6 digits)
5. Send email via EmailJS
6. Store user in database with verified=0
7. Return success (user must verify email)

Password hashing (code line 45-50):

```
pwd_context = CryptContext(
    schemes=["bcrypt_sha256", "pbkdf2_sha256"],
    default="bcrypt_sha256",
    bcrypt_sha256__rounds=12  # From config.py
)
```

B. Email Verification (POST /verify-otp)

Code location: Line 1162-1219

User enter OTP code from email. If match:

- Set verified=1 in database
- Create session token
- Set cookie
- Redirect to chat page

C. User Login (POST /login)

Code location: Line 1399-1508

Security features implemented:

1. **Rate limiting**: 5 attempts per minute per IP
2. **Account lockout**: After 5 failed attempts, lock account for 15 minutes
3. **Session creation**: Generate secure token with session_id
4. **Password upgrade**: If user have old pbkdf2_sha256 hash, auto-upgrade to bcrypt_sha256 on successful login

Code for password verification (line 1460-1475):

```
if not pwd_context.verify(password, user_row[2]):  # user_row[2] is password hash
    record_failed_login(username, client_ip)
    raise HTTPException(status_code=401, detail="Invalid credentials")

# Check if password needs rehashing (upgrade from pbkdf2 to bcrypt)
if pwd_context.needs_update(user_row[2]):
    new_hash = pwd_context.hash(password)
    cursor.execute("UPDATE users SET password=? WHERE username=?",
                   (new_hash, username))
    conn.commit()
```

D. Google OAuth (GET /auth/google/login)

Code location: Line 1307-1384

Flow:

1. User click "Sign in with Google"
2. Redirect to Google consent page
3. Google redirect back to `/auth/google/callback`
4. Exchange code for access token
5. Get user info from Google
6. Check if user exist in database:
    - If yes: Log them in
    - If no: Create new user with auth_provider="google"
7. Create session and redirect to chat

OAuth configuration (config.py):

```
GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
GOOGLE_CLIENT_SECRET = os.getenv("GOOGLE_CLIENT_SECRET")
GOOGLE_REDIRECT_URI = "http://localhost:7001/auth/google/callback"
```

E. WebSocket Proxy (WebSocket /ws)

Code location: Line 3970-4061

Why need proxy?

- Frontend connect to login server (port 7001)
- But RAG server run on port 7000
- Can't connect directly due to session cookie on different port

How it work:

1. Client connect WebSocket to login server /ws
2. Login server create WebSocket to RAG server
3. Forward all messages bidirectionally:
    - Client → Login → RAG
    - RAG → Login → Client
4. When either side disconnect, close both connections

Code logic:

```
async def forward_from_rag(rag_ws, client_ws):
    async for msg in rag_ws:
        if msg.type == aiohttp.WSMsgType.TEXT:
            await client_ws.send_text(msg.data)
        elif msg.type == aiohttp.WSMsgType.BINARY:
            await client_ws.send_bytes(msg.data)

async def forward_from_client(client_ws, rag_ws):
    async for msg in client_ws.iter_text():
        await rag_ws.send_str(msg)
```

F. Admin Panel Features

Implemented admin pages:

1. **User Management** (GET /admin/users)
    - View all registered user
    - Delete user account
    - View user role (admin/employee/customer)
2. **Knowledge Base Management** (GET /admin/knowledge)
    - Upload document (TXT or PDF)
    - View all document in ChromaDB
    - Count total document
3. **Analytics Dashboard** (GET /admin/analytics)
    - Total queries processed
    - Average response time
    - Success vs error rate
    - Popular queries
4. **Error Logs** (GET /admin/errors)
    - View all error that happen
    - See error message, timestamp, user

All admin pages require authentication:

```
user = Depends(require_login(role="admin"))
```

---

3. Database Schema (SQLite3)

3.1 Users Database (users.db)

Location: Login server manages this

Tables:

users

```
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,           -- bcrypt hash
    email TEXT UNIQUE NOT NULL,
    role TEXT DEFAULT 'customer',     -- admin/employee/customer
```

```
    verified INTEGER DEFAULT 0,       -- 0=not verified, 1=verified
    auth_provider TEXT DEFAULT 'local', -- local/google
    otp_code TEXT,                    -- For email verification
    otp_expiry DATETIME,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_login DATETIME
);
```

sessions (stored in memory)

```
user_sessions = defaultdict(list)  # user_id -> list of session info
invalidated_sessions = set()       # Set of revoked session IDs
```

Note: Session not in database, stored in RAM. Will reset when server restart. In production should use Redis.

3.2 Conversations Database (conversations.db)

Location: RAG server manages this

Tables:

conversations

```
CREATE TABLE conversations (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    connection_id TEXT NOT NULL,      -- Like "conn_a3f2b891"
    user_message TEXT NOT NULL,
    ai_response TEXT NOT NULL,
    retrieved_docs TEXT,              -- Documents found by RAG (newline
separated)
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

sessions

```
CREATE TABLE sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    connection_id TEXT UNIQUE NOT NULL,
    start_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    end_time DATETIME,
    message_count INTEGER DEFAULT 0
);
```

Functions (from database.py):

- `init_database()`: Create tables if not exist
- `save_conversation(connection_id, user_msg, ai_msg, docs)`: Save one Q&A
- `start_session(connection_id)`: Start new session
- `end_session(connection_id)`: Mark session as ended
- `get_stats()`: Get statistics (total messages, avg per session)

3.3 Analytics Database (analytics.db)

Location: RAG server manages this

Tables:

usage_logs

```sql
CREATE TABLE usage_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    user_role TEXT,
    query_text TEXT,
    query_length INTEGER,
    response_time_ms INTEGER,        -- Milliseconds
    rag_docs_found INTEGER,
    success INTEGER,                 -- 1=success, 0=error
    error_message TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Functions (from analytics.py):

- `init_analytics_db()`: Create table
- `log_usage(username, role, query, response_time, docs_count, success, error)`: Log each request

Analytics queries:

```sql
# Total queries
SELECT COUNT(*) FROM usage_logs

# Average response time
SELECT AVG(response_time_ms) FROM usage_logs WHERE success=1

# Success rate
SELECT (SUM(success) * 100.0 / COUNT(*)) FROM usage_logs

# Popular queries
SELECT query_text, COUNT(*) as count FROM usage_logs
GROUP BY query_text ORDER BY count DESC LIMIT 10
```

---

4. TOON Format Implementation

4.1 What is TOON?

TOON = **Token-Oriented Object Notation**

Problem it solve:

- JSON use many characters: `{"key": "value"}` = 17 characters
- LLM count tokens, more tokens = slower + more expensive
- For RAG context, we send many documents to LLM
- JSON waste tokens on `{`, `}`, `"`, `:`, etc.

TOON solution:

- Minimal syntax
- Flat structure (no nesting)
- Arrays use length prefix
- Save 40-60% tokens

4.2 Format Specification (from toon.py)

Simple value:

```
key: value
```

Array:

```
key[length]: item1,item2,item3
```

Nested object (flattened):

```
parent.child: value
```

Example conversion:

JSON (85 tokens):

```
{
  "documents": [
    {
      "page_content": "To reset password, click Forgot Password link.",
      "metadata": {
        "doc_id": 0,
        "type": "support_info"
```

```
        }
      }
    ]
  }
```

TOON (48 tokens):

```
doc[1]:
content: To reset password, click Forgot Password link.
type: support_info
```

Token savings: (85 - 48) / 85 = 43.5%

4.3 Implementation Code

Function: to_toon()

Location: `backend/toon.py` line 18-62

```python
def to_toon(data: dict, prefix: str = "") -> str:
    lines = []

    for key, value in data.items():
        full_key = f"{prefix}.{key}" if prefix else key

        if value is None:
            continue  # Skip None to save tokens
        elif isinstance(value, dict):
            nested_toon = to_toon(value, prefix=full_key)
            if nested_toon:
                lines.append(nested_toon)
        elif isinstance(value, list):
            if len(value) == 0:
                lines.append(f"{full_key}[0]:")
            else:
                items_str = ','.join(str(item) for item in value)
                lines.append(f"{full_key}[{len(value)}]: {items_str}")
        elif isinstance(value, bool):
            lines.append(f"{full_key}: {1 if value else 0}")  # bool as 0/1
        else:
            lines.append(f"{full_key}: {value}")

    return '\n'.join(lines)
```

Function: from_toon()

Location: `backend/toon.py` line 65-130

Convert TOON back to dictionary (for parsing response if needed).

Function: format_rag_context_toon()

Location: `backend/toon.py` line 200-250

Special function for RAG context:

```
def format_rag_context_toon(documents: list) -> str:
    """Format RAG documents in TOON for minimal token usage"""
    if not documents:
        return ""

    lines = [f"doc[{len(documents)}]:"]

    for i, doc in enumerate(documents):
        content = doc.get("page_content", "")
        lines.append(f"content: {content}")

        # Add metadata if exists
        metadata = doc.get("metadata", {})
        if metadata:
            for key, val in metadata.items():
                if val is not None:
                    lines.append(f"{key}: {val}")

        if i < len(documents) - 1:
            lines.append("---")  # Separator between docs

    return '\n'.join(lines)
```

Usage in RAG pipeline:

```
# Instead of this (JSON):
context = json.dumps({"documents": relevant_docs})

# We do this (TOON):
context = format_rag_context_toon(relevant_docs)
```

Benefit measured:

- Test case: 3 documents, each 200 characters
- JSON format: 850 tokens
- TOON format: 340 tokens
- Savings: 60%
- Speed improvement: ~15 seconds faster response

---

5. Security Implementation (OWASP)

5.1 Actually Implemented Security (Not Just Theory)

A. Password Security

Hashing algorithm (code line 45-50 in login_server.py):

```
pwd_context = CryptContext(
    schemes=["bcrypt_sha256", "pbkdf2_sha256"],
    default="bcrypt_sha256",           # Current standard
    deprecated=["pbkdf2_sha256"],      # Old hash, auto-upgrade
    bcrypt_sha256__rounds=12           # Cost factor
)
```

Why bcrypt_sha256?

- bcrypt have 72 byte limit, bcrypt_sha256 fix this by hash password with SHA256 first
- Slow by design (prevent brute force)
- 12 rounds = ~300ms per hash (good balance)

Password validation (code line 1060-1070):

```
# Regex pattern for strong password
PASSWORD_PATTERN = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?
&]{8,128}$'

if not re.match(PASSWORD_PATTERN, password):
    raise HTTPException(
        status_code=400,
        detail="Password must be 8-128 characters with uppercase, lowercase,
digit, and special character"
    )
```

Auto-upgrade old password (code line 1460-1475):

```
# After successful login
if pwd_context.needs_update(user_row[2]):
    new_hash = pwd_context.hash(password)
    cursor.execute("UPDATE users SET password=? WHERE username=?",
                   (new_hash, username))
    conn.commit()
```

B. Rate Limiting

Implementation (code line 217-245):

```
rate_limit_store = defaultdict(lambda: {"count": 0, "reset_time": time.time()})

def check_rate_limit(identifier: str, limit: int, window_seconds: int = 60) ->
bool:
    now = time.time()

    if now > rate_limit_store[identifier]["reset_time"]:
        # Reset window
        rate_limit_store[identifier] = {
            "count": 1,
            "reset_time": now + window_seconds
        }
        return True

    if rate_limit_store[identifier]["count"] >= limit:
        return False  # Rate limit exceeded

    rate_limit_store[identifier]["count"] += 1
    return True
```

Where applied:

- Login: 5 requests/minute per IP
- Register: 3 requests/minute per IP
- OTP: 3 requests/minute per IP

Usage:

```
client_ip = request.client.host
if not check_rate_limit(f"login:{client_ip}", RATE_LIMIT_LOGIN):
    raise HTTPException(status_code=429, detail="Too many requests")
```

Note: This is in-memory, reset when server restart. In production should use Redis.

C. Account Lockout

Implementation (code line 247-290):

```
failed_login_attempts = {}  # username -> count
account_lockouts = {}       # username -> lockout_until_timestamp
MAX_FAILED_ATTEMPTS = 5
LOCKOUT_DURATION = 900      # 15 minutes in seconds

def record_failed_login(username: str, ip_address: str):
    if username not in failed_login_attempts:
        failed_login_attempts[username] = 0

    failed_login_attempts[username] += 1
```

```
    if failed_login_attempts[username] >= MAX_FAILED_ATTEMPTS:
        account_lockouts[username] = time.time() + LOCKOUT_DURATION
        log_security_event("account_lockout", {
            "username": username,
            "ip_address": ip_address,
            "lockout_duration_seconds": LOCKOUT_DURATION
        }, severity="CRITICAL")
```

Before login check:

```
is_locked, remaining = check_account_lockout(username)
if is_locked:
    raise HTTPException(
        status_code=403,
        detail=f"Account locked. Try again in {remaining} seconds."
    )
```

D. Session Security

Session token creation (code line 292-350):

```
def create_session_token(username: str, role: str, **extra) -> str:
    session_id = secrets.token_urlsafe(32)  # Cryptographically secure
    now = time.time()

    session_data = {
        "username": username,
        "role": role,
        "session_id": session_id,
        "created_at": now,
        "last_activity": now,
        **extra
    }

    return serializer.dumps(session_data)  # Sign with SECRET_KEY
```

Session validation (code line 352-380):

```
def validate_session(session_data: dict) -> tuple[bool, str]:
    # Check if invalidated (logout)
    if session_data.get("session_id") in invalidated_sessions:
        return False, "Session invalidated"

    # Check absolute timeout (24 hours)
    created_at = session_data.get("created_at", 0)
    if time.time() - created_at > SESSION_ABSOLUTE_TIMEOUT:
```

```
        return False, "Session expired (absolute)"

    # Check idle timeout (2 hours)
    last_activity = session_data.get("last_activity", created_at)
    if time.time() - last_activity > SESSION_IDLE_TIMEOUT:
        return False, "Session expired (idle)"

    return True, ""
```

Cookie settings:

```
response.set_cookie(
    key="session",
    value=token,
    httponly=True,      # Prevent XSS access
    secure=False,       # True in production (HTTPS only)
    samesite="lax",     # CSRF protection
    max_age=86400       # 24 hours
)
```

E. SQL Injection Prevention

All database queries use parameterized queries:

```
# WRONG (vulnerable):
cursor.execute(f"SELECT * FROM users WHERE username='{username}'")

# CORRECT (safe):
cursor.execute("SELECT * FROM users WHERE username=?", (username,))
```

Example from code (line 1420):

```
cursor.execute(
    "SELECT id, username, password, role, verified, auth_provider FROM users WHERE
username=?",
    (username,)
)
```

The `?` placeholder prevent SQL injection because value is escaped by SQLite driver.

F. XSS Prevention

Output escaping in Jinja2 templates:

Jinja2 auto-escape HTML by default:

```
<!-- User input automatically escaped -->
<p>Welcome {{ username }}</p>

<!-- If username = "<script>alert('XSS')</script>" -->
<!-- Rendered as: <p>Welcome &lt;script&gt;alert('XSS')&lt;/script&gt;</p> -->
```

Content Security Policy header:

```
# In production, add CSP header
response.headers["Content-Security-Policy"] = "default-src 'self'; script-src
'self'"
```

Note: CSP not fully implemented yet. Should add in production.

G. CSRF Protection

Implementation (code line 221-225 in assistify_rag_server.py):

```
def verify_csrf(request: Request):
    csrf_header = request.headers.get("x-csrf-token")
    csrf_cookie = request.cookies.get("csrf_token")
    if not csrf_cookie or csrf_header != csrf_cookie:
        raise HTTPException(status_code=403, detail="CSRF token missing or
invalid")
```

Usage on state-changing endpoints:

```
@app.post("/upload")
async def upload_document(request: Request, ...):
    verify_csrf(request)  # Check CSRF before processing
    # ... rest of code
```

How it work:

1. Server set CSRF token in cookie
2. Frontend read cookie and send in header
3. Server compare cookie value with header value
4. If not match = CSRF attack, reject request

Note: Not all endpoint have CSRF check yet. Need to add to all POST/PUT/DELETE.

H. Input Validation

Using Pydantic models (code line 900-950):

```python
class RegisterRequest(BaseModel):
    username: constr(min_length=3, max_length=20, regex=r'^[a-zA-Z0-9_]+$')
    password: constr(min_length=8, max_length=128)
    email: EmailStr
    role: constr(regex=r'^(admin|employee|customer)$') = 'customer'

    @validator('password')
    def validate_password(cls, v):
        if not re.match(PASSWORD_PATTERN, v):
            raise ValueError('Weak password')
        return v
```

This prevent:

- Username with special characters (SQL injection)
- Short/long passwords
- Invalid email format
- Invalid role values

I. Security Logging

All security event logged (code line 52-85):

```python
def log_security_event(event_type: str, details: dict, severity: str = "INFO"):
    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "event_type": event_type,
        "severity": severity,
        **details
    }
    security_logger.info(json.dumps(event))
```

Events logged:

- login_success
- login_failure
- account_lockout
- session_created
- session_invalidated
- otp_sent
- otp_verified
- user_registered
- password_changed
- unhandled_exception

Log file: `logs/security.log` (rotating, max 10MB × 5 files)

Example log entry:

```
{
  "timestamp": "2025-11-24T10:30:45.123456",
  "event_type": "login_failure",
  "severity": "WARNING",
  "username": "testuser",
  "ip_address": "192.168.1.100",
  "attempt_count": 3
}
```

5.2 Security Gaps (Honest Assessment)

What NOT implemented yet:

1. **HTTPS/TLS**: Running on HTTP (should be HTTPS in production)
2. **CSRF on all endpoint**: Only some endpoint have CSRF check
3. **Content Security Policy**: Header not set
4. **Redis for session**: Using in-memory storage (reset on restart)
5. **Two-Factor Authentication**: Started but not complete
6. **File upload validation**: Accept any TXT/PDF without scanning for malware
7. **API rate limiting per user**: Only per IP, not per authenticated user
8. **Password history**: User can reuse old password
9. **Audit log for admin action**: No log when admin delete user
10. **Input size limit**: No limit on uploaded file size

---

6. API Endpoints Reference

6.1 LLM Server (Port 8000)

GET /health

Check if LLM ready.

Response:

```
{
  "status": "ready",
  "model": "qwen2.5-7b-instruct",
  "gpu_layers": 10,
  "context_size": 512
}
```

POST /v1/chat/completions

Generate response from LLM.

Request:

```
{
  "model": "qwen2.5-7b-instruct",
  "messages": [
    {"role": "system", "content": "You are helpful assistant"},
    {"role": "user", "content": "What is RAG?"}
  ],
  "max_tokens": 80,
  "temperature": 0.7,
  "stop": null
}
```

Response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1700000000,
  "model": "qwen2.5-7b-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "RAG stands for Retrieval-Augmented Generation..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 25,
    "completion_tokens": 45,
    "total_tokens": 70
  }
}
```

6.2 RAG Server (Port 7000)

POST /query

Send text query (no voice).

Request:

```
POST /query
Content-Type: application/x-www-form-urlencoded
```

```
text=How to reset password?
```

Response:

```
{
  "response": "To reset password, click Forgot Password link on login page...",
  "sources": 1
}
```

Auth: Required (must be logged in)

POST /upload

Upload document to knowledge base.

Request:

```
POST /upload
Content-Type: multipart/form-data

file: [document.pdf binary data]
```

Response:

```
{
  "message": "File document.pdf uploaded and indexed as
upload_a3f2b891_document.pdf."
}
```

Auth: Required (admin only)

WebSocket /ws

Real-time voice chat.

Message from client:

```
// Audio chunk
Binary: [PCM16 audio data]

// Ping
{"type": "ping"}
```

```
// Stop recording
{"type": "control", "action": "stop_recording"}
```

Message from server:

```
// Transcript
{
  "type": "transcript",
  "text": "How to reset password",
  "final": true
}

// AI response
{
  "type": "aiResponse",
  "text": "To reset password, click...",
  "sources": 1
}

// Pong
{"type": "pong"}
```

6.3 Login Server (Port 7001)

POST /register

Register new user.

Request:

```
{
  "username": "testuser",
  "password": "SecurePass123!",
  "email": "test@example.com",
  "role": "customer"
}
```

Response:

```
{
  "message": "Registration successful. Check email for OTP."
}
```

POST /verify-otp

Verify email with OTP code.

Request:

```
POST /verify-otp
Content-Type: application/x-www-form-urlencoded

username=testuser&otp_code=123456
```

Response:

```
302 Redirect to /customer
Set-Cookie: session=...
```

POST /login

Login with username/password.

Request:

```
POST /login
Content-Type: application/x-www-form-urlencoded

username=testuser&password=SecurePass123!
```

Response (success):

```
302 Redirect to /customer (or /admin, /employee based on role)
Set-Cookie: session=...
```

Response (failure):

```
{
  "detail": "Invalid credentials"
}
```

Status: 401

GET /auth/google/login

Start Google OAuth flow.

Response:

```
302 Redirect to Google consent page
```

GET /auth/google/callback

Google OAuth callback.

Query params:

```
code=abc123xyz...
state=random_state
```

Response:

```
302 Redirect to /customer
Set-Cookie: session=...
```

GET /logout

Logout and invalidate session.

Response:

```
302 Redirect to /login
Set-Cookie: session=; Max-Age=0
```

GET /admin/users

View all users (admin only).

Response:

```
<html>
  <table>
    <tr><th>ID</th><th>Username</th><th>Email</th><th>Role</th></tr>
    <tr><td>1</td><td>admin</td><td>admin@example.com</td><td>admin</td></tr>
    ...
  </table>
</html>
```

GET /admin/analytics

View usage statistics (admin only).

Response:

```
<html>
  <h2>Analytics Dashboard</h2>
  <p>Total Queries: 1,234</p>
  <p>Average Response Time: 12.5s</p>
  <p>Success Rate: 98.5%</p>
  ...
</html>
```

WebSocket /ws

Proxy to RAG server.

Just forward all message to RAG server WebSocket.

---

7. How to Run System (Step by Step)

7.1 Prerequisites

Hardware:

- NVIDIA GPU with 8GB+ VRAM
- 16GB+ RAM
- 20GB free disk space

Software:

- Windows 10/11
- Python 3.10 or 3.11 (not 3.12, some package not compatible)
- NVIDIA Driver (latest)
- CUDA Toolkit 12.x
- Git

7.2 Installation Steps

1. **Clone repository**

   ```
   cd C:\Users\Jonathan\Desktop\AAST
   # Already cloned as "Graduation Project"
   ```

2. **Install Python dependencies**

```
cd "Graduation Project"
pip install -r requirements.txt
```

3. **Install llama-cpp-python with CUDA**

```
$env:CMAKE_ARGS="-DLLAMA_CUBLAS=ON"
pip install llama-cpp-python --force-reinstall --no-cache-dir
```

4. **Create `.env` file**

```
ni .env
```

Content:

```
SESSION_SECRET=your_64_byte_secret_here_change_this_in_production_min_length
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
EMAILJS_PUBLIC_KEY=your_emailjs_public_key
EMAILJS_PRIVATE_KEY=your_emailjs_private_key
EMAILJS_SERVICE_ID=your_emailjs_service_id
EMAILJS_TEMPLATE_ID=your_emailjs_template_id
```

5. **Download model** (if not exist)

Model already in `backend/Models/Qwen2.5-7B-LLM/`, check it exist:

```
ls backend\Models\Qwen2.5-7B-LLM\
```

Should see 2 GGUF files (total ~4.3GB)

6. **Initialize knowledge base** (optional)

```
python backend\load_documents.py
```

7.3 Start Servers

Option 1: Use batch script (recommended)

```
.\scripts\start_all_servers.bat
```

This will:

1. Kill existing process on port 8000, 7000, 7001
2. Start LLM server (wait 15s)
3. Start RAG server (wait 20s)
4. Start Login server (wait 5s)
5. Total wait: ~45 seconds

Option 2: Manual start (for debugging)

Terminal 1 - LLM Server:

```
cd backend
python main_llm_server.py
# Wait until see "✓ Model loaded successfully" (~18 seconds)
```

Terminal 2 - RAG Server:

```
cd backend
python assistify_rag_server.py
# Wait until see "✓ faster-whisper loaded successfully" (~10 seconds)
```

Terminal 3 - Login Server:

```
cd Login_system
python login_server.py
# Should start immediately
```

7.4 Access System

1. Open browser
2. Go to: `http://localhost:7001`
3. Click "Register" to create account
4. Check email for OTP code
5. Enter OTP to verify
6. Login with username/password
7. You will see chat interface
8. Click microphone to start voice chat
9. Or type message in text box

---

8. Performance Metrics (Actual Measurements)

8.1 Server Startup Time

LLM Server:

- Load model from disk: ~18 seconds
- Initialize GPU: ~2 seconds
- Total: ~20 seconds

RAG Server:

- Initialize databases: ~1 second
- Load faster-whisper: ~10 seconds
- Total: ~11 seconds

Login Server:

- Start immediately: <1 second

Total system startup: ~45 seconds (using batch script with wait time)

8.2 Response Time

Greeting (without RAG):

- User say "hi"
- Voice transcription: 0.5s
- Greeting detection: 0.01s
- LLM inference: 2.8s
- Total: ~3.3s

RAG Query:

- User ask "How to reset password?"
- Voice transcription: 0.8s
- ChromaDB search: 0.3s
- TOON formatting: 0.01s
- LLM inference: 10.5s (with context)
- Total: ~11.6s

Old performance (before optimization):

- Same RAG query: 50 seconds
- Problem: 3 documents, long system prompt, 200 max tokens, JSON format

Optimization result:

- 76% faster (50s → 11.6s)
- Achieved by: Greeting detection, top_k=1, TOON format, 80 max tokens, minimal prompt

8.3 GPU Memory Usage

Idle (model loaded):

- GPU: 1,353 MiB (model layers)

- System RAM: 3,713 MiB (model weights not on GPU)

During inference:

- GPU: +200-400 MiB (depends on context size)
- Peak: ~1,600 MiB
- Safe under 2GB limit for 10 GPU layers

During voice transcription:

- faster-whisper: +800 MiB
- Total GPU: ~2,200 MiB
- Still safe

Why only 10 layers on GPU?

- Full model = 32 layers
- All layers on GPU = ~7GB (too much for 8GB GPU)
- 10 layers = good balance speed vs memory
- CPU handle other 22 layers

8.4 Token Usage (with TOON)

Test case: 3 documents RAG context

Format	Tokens	Savings
JSON	850	-
TOON	340	60%

Impact on response time:

- More tokens = slower inference
- 850 tokens ≈ 27s to process
- 340 tokens ≈ 10s to process
- Saved: 17 seconds per request

---

9. Known Issues and Limitations

9.1 Current Bugs

1. **Voice recording freeze** if user speak too long (>30 seconds)
    - Cause: Buffer overflow
    - Workaround: Pause between sentences
2. **Session lost on server restart**
    - Cause: In-memory session storage
    - Impact: All user logged out
    - Solution: Need implement Redis

3. **PDF parsing fail** for scanned PDF
    - Cause: PyPDF2 only extract text layer
    - Impact: Scanned document not indexed
    - Solution: Need implement OCR (tesseract)
4. **ChromaDB lock error** if multiple upload same time
    - Cause: SQLite lock in ChromaDB
    - Impact: Upload fail with error
    - Solution: Add upload queue
5. **CUDA out of memory** if ask very long question
    - Cause: Context window overflow
    - Impact: Server crash
    - Solution: Add input length validation (done, max 1000 chars)

9.2 Performance Limitation

1. **Only 1 concurrent user** on voice
    - Reason: GPU can only run 1 inference at time
    - Solution: Need implement queue system
2. **Response time vary** (10-20 seconds)
    - Depend on: Question length, RAG docs found, GPU temperature
    - Cannot predict exact time
3. **Model limited knowledge** (training data cutoff October 2023)
    - Cannot answer question about recent event
    - Solution: Keep knowledge base updated

9.3 Feature Not Implemented

These feature in documentation but not in code:

1. **Multi-language support** - Only English work
2. **Voice output customization** - Use browser TTS, no control
3. **Conversation export** - Cannot download chat history
4. **User profile page** - Can only change password
5. **Real-time typing indicator** - Not implemented
6. **File attachment in chat** - Only admin can upload document
7. **Mobile responsive** - UI not optimized for phone
8. **Dark mode** - Only light theme available

---

10. Future Improvement Plan

10.1 Performance

- ☐ Implement request queue for concurrent user
- ☐ Cache frequent queries
- ☐ Pre-compute document embeddings
- ☐ Use Redis for session storage
- ☐ Implement WebSocket compression
- ☐ Add CDN for static files

10.2 Features

- ☐ Add conversation export (PDF/TXT)
- ☐ Implement user profile page
- ☐ Add file attachment in chat
- ☐ Multi-language support (Arabic, French)
- ☐ Real-time typing indicator
- ☐ Voice output customization
- ☐ Mobile app (React Native)

10.3 Security

- ☐ Implement HTTPS/TLS
- ☐ Add CSRF to all endpoints
- ☐ Set Content Security Policy
- ☐ File upload antivirus scan
- ☐ Complete Two-Factor Authentication
- ☐ API rate limiting per user
- ☐ Password history check
- ☐ Admin action audit log

10.4 Scalability

- ☐ Move to PostgreSQL (from SQLite)
- ☐ Implement Redis for cache
- ☐ Use message queue (RabbitMQ)
- ☐ Deploy on cloud (AWS/Azure)
- ☐ Load balancer for multiple instance
- ☐ GPU cluster for LLM inference

---

11. Deployment Guide (Production)

11.1 Environment Variables (Required)

Create `.env` file with these value:

```
# Environment
ENVIRONMENT=production

# Security (MUST change these)
SESSION_SECRET=[64+ random bytes]
```

```
ENFORCE_HTTPS=true
ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com

# OAuth
GOOGLE_CLIENT_ID=[from Google Cloud Console]
GOOGLE_CLIENT_SECRET=[from Google Cloud Console]
GOOGLE_REDIRECT_URI=https://yourdomain.com/auth/google/callback

# Email
EMAILJS_PUBLIC_KEY=[from EmailJS]
EMAILJS_PRIVATE_KEY=[from EmailJS]
EMAILJS_SERVICE_ID=[from EmailJS]
EMAILJS_TEMPLATE_ID=[from EmailJS]

# Rate Limiting
RATE_LIMIT_LOGIN=5
RATE_LIMIT_REGISTER=3
RATE_LIMIT_OTP=3

# Password Hashing
BCRYPT_ROUNDS=12

# URLs
BASE_URL=https://yourdomain.com
LLM_SERVER_URL=http://localhost:8000
RAG_SERVER_URL=http://localhost:7000
```

11.2 Reverse Proxy (Nginx)

```
server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    # Login server
    location / {
        proxy_pass http://localhost:7001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # WebSocket
    location /ws {
        proxy_pass http://localhost:7001/ws;
```

```
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 86400;  # 24 hours
    }
}
```

11.3 Process Manager (systemd)

Create `/etc/systemd/system/assistify-llm.service`:

```
[Unit]
Description=Assistify LLM Server
After=network.target

[Service]
Type=simple
User=assistify
WorkingDirectory=/opt/assistify/backend
Environment="PATH=/opt/assistify/venv/bin"
ExecStart=/opt/assistify/venv/bin/python main_llm_server.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Repeat for RAG and Login server.

Start:

```
sudo systemctl enable assistify-llm assistify-rag assistify-login
sudo systemctl start assistify-llm assistify-rag assistify-login
```

---

12. Conclusion

This documentation describe actual implementation of Assistify system based on real code. Not theory or plan, but what actually working right now.

Main achievement:

- 3-tier architecture with separate concern
- Voice input using faster-whisper on GPU
- RAG system with ChromaDB and semantic search
- TOON format innovation (40-60% token savings)
- Complete authentication with OAuth and OTP
- Security implement from OWASP top 10

- Performance optimization (50s → 11s response time)

Limitation acknowledged:

- Only 1 concurrent user for voice
- Session lost on restart (in-memory)
- Some security gap exist
- Some feature in plan not implement yet

Ready for production? Not yet. Need:

- Redis for session
- HTTPS with valid certificate
- More security hardening
- Queue system for concurrent user
- Better error handling
- More testing

But work good for demonstration and graduation project presentation.

Document end. All information based on actual code in repository.